

CMPT 863 G100

Group Project Part 3

User Interface Prototypes (Iteration 1)

William Fung #301329660

Vivek Pisharodi #301620021

Jin Niu #301396123

Overview

The goal of this prototype was to design a mobile shopping list application — **ShopBuddy** — that helps users quickly create, manage, and optimize their grocery shopping. The design emphasizes simplicity and efficiency so that users can easily manage their shopping tasks before, during, and after a shopping trip. Both prototypes were built in Figma and cover the primary user flows: onboarding, managing shopping lists, adding items, scanning receipts, comparing prices across stores, and configuring AI-driven shopping rules.

Part I: Horizontal Prototype

Design Rationale

The horizontal prototype covers the full breadth of ShopBuddy's features across three core sections, accessible at all times via a persistent bottom navigation bar: **Lists**, **Receipt**, and **Settings**.

The navigation labels themselves evolved from Part 2. Prototype #4 (our most promising paper prototype) used the labels "New List," "Receipt Upload," and "Settings." In the horizontal prototype, we renamed "New List" to "Lists" to better reflect that this tab manages all existing lists — not just list creation — and shortened "Receipt Upload" to "Receipt" for brevity. This shift aligns with standard mobile navigation conventions, where tab labels describe the section rather than a single action.

Onboarding was a new addition absent from all Part 2 prototypes. Users complete a 5-step flow covering shopping preferences (in-person vs. online), personal information, and EULA acceptance before reaching the main interface. This lets the AI personalize recommendations from the first session, directly addressing the user need raised in our interviews: participants wanted the app to "learn" their preferences rather than starting from scratch each time.

The Lists tab (home screen) introduces a personalized greeting and two AI quick-action prompts (e.g., *"Build a weekly list for 2"*). These serve as a low-friction entry point for AI interaction, giving users immediate suggestions without requiring them to open a list first. Below the prompts, all shopping lists are displayed as cards showing item count and last-modified date, providing an at-a-glance context.

The AI chatbox — present in Prototype #3 as a global element — is retained and appears on both the home screen and within each List Detail screen as a *"What would you like to do?"* input. On the home screen, it pairs with two pre-filled quick-action prompts to guide users who are unsure what to type; within a list, it accepts context-specific requests such as adding items or adjusting quantities. This dual placement ensures the AI is accessible at the top-level planning stage as well as during the detailed in-list editing stage.

List Detail (reached by tapping any list) groups items by category (Dairy, Meat, Produce), displays quantities, and highlights sale items with a badge. This addresses the finding from P2, who wanted items organized so that in-store navigation is easier. Users can check off items, adjust quantities, add new items via a categorized form with AI-suggested items from purchase history, or initiate a receipt scan. A floating AI prompt within the list allows natural-language updates (e.g., *"Add ingredients for guacamole"*).

Price Comparison was conceptualized in Prototype #3 but was not implemented. The horizontal prototype realizes this fully: a sortable table compares prices for every item across Walmart, Superstore, and Safeway, with per-item and total costs shown side by side. Users can either review the table manually or tap **Choose For Me (Cheapest)** to delegate the decision to the AI.

AI-Optimized Checkout is a new feature not present in Part 2. After the price comparison, the AI selects the cheapest cross-store combination (e.g., dairy from Superstore, meat from Walmart) and displays the savings. Each store section links directly to that retailer's website. This directly solves the core problem statement: reducing manual effort while saving money.

List Sharing (classified as "Could include" in the analysis in Part 2) was promoted to implementation after interviews confirmed that household coordination is a key pain point. Users can invite others by email and assign Viewer or Editor roles, matching the mental model of tools like Google Docs that participants currently use.

Settings expanded from a single screen in Part 2 into six subcategories: Dietary Preferences, Brand Choices, Budget Limits, Order Rules, Price Comparison preferences, and Store Preferences. The **Order Rules** screen, introduced conceptually in Prototype #2, is now fully shown: users enter plain-language rules at both a global level (e.g., *"Prefer organic for all produce"*) and per-list (e.g., *"Budget limit: \$80 per trip"*), which are passed as context to the AI.

Part II: Vertical Prototype

Design Rationale

The vertical prototype is implemented within the same Figma file as the horizontal prototype, focusing on the complete depth of the application's core user journey: creating and managing a shopping list, comparing prices across stores, and checking out via AI-optimized store selection. This flow was chosen because it represents the most unique and differentiating aspect of ShopBuddy — the end-to-end path from list to purchase decision — and directly maps to the primary tasks identified in our user interviews.

Starting from the **Lists home screen**, the vertical flow proceeds through four connected, interactive screens:

1. **List Detail (Weekly Groceries)** — Items are grouped by collapsible category (Dairy, Meat, Produce, Bakery). Each item displays its name, unit metric (e.g., "4L," "1kg"), a quantity stepper, and contextual badges: SALE for discounted items, OUT for unavailable items with an inline substitute suggestion (e.g., "↔ Guacamole Tub" beside unavailable Avocados). Users check items off with visual strikethrough feedback. Two action buttons at the bottom ("Compare Prices" and "Checkout") advance the flow.
2. **Price Comparison** — A table compares every unchecked item across Walmart, Superstore, and Safeway, with the cheapest option per item highlighted. Per-store totals appear at the bottom. Users proceed via "Choose For Me (Cheapest)" for AI optimization or "Manual Checkout" for direct control.
3. **AI-Optimized Checkout** — A savings banner (e.g., "Saving you ~\$8.47") is followed by store-grouped item cards with color-coded headers and per-store "Go to [Store] →" redirect buttons.

This vertical slice was deliberately scoped to exclude login flows and settings, as the assignment guidance recommends focusing on features that are most unique to the product. Price comparison and AI-optimized checkout are the features that most directly differentiate ShopBuddy from generic shopping list apps, and they are the features our interview participants most consistently requested. They were also the most consistently requested features across all four interview participants: P1 and P2 both emphasized finding discounts and comparing prices, P3's primary concern was price comparison across stores, and P4 explicitly wanted the application to show the lowest available

prices. By making this flow the vertical focus, we ensure the prototype demonstrates the exact functionality that our user research independently validated as most valuable.

Compared to Part 2's Prototype #3 (which described the price comparison and "Choose for me" concept but left it unimplemented), and Prototype #4 (which outlined the three-tab structure without any checkout flow), the vertical prototype delivers a fully interactive, end-to-end realization of these features with realistic data, meaningful interactions, and a clear sense of what the finished experience would feel like.

Strengths and Areas for Improvement

What Works Well

- **Persistent bottom navigation** keeps all three core sections accessible at all times, reducing navigation overhead especially during in-store use.
- **Price comparison table with AI-optimized checkout** is our most differentiating feature. It directly addresses the primary user need (saving money with minimal effort) identified across all four interview participants.
- **Plain-language order rules** leverage the strengths of the underlying LLM and give users meaningful, lasting control over AI behavior without requiring them to understand a formal rule syntax.
- **List sharing with Viewer/Editor roles** supports the household coordination use case in a familiar, low-friction way.

Key Areas for Improvement

1. **In-store aisle sorting.** P2 explicitly asked for items to be sortable by store section to reduce backtracking while shopping. The current design organizes items by food category (Dairy, Meat, Produce), which is a good starting point, but does not map to physical store layouts, which vary by retailer. A future iteration should allow the app to reorder items based on the user's chosen store's aisle map.
2. **AI response visibility and error recovery.** The AI textbox and suggested prompts are present throughout the prototype, but the current design does not yet show what happens after the user submits a request — how the AI confirms understanding, previews proposed changes, or allows the user to undo an action. A future iteration should add a lightweight confirmation step (e.g., "I'll add 6 items for guacamole — confirm?") and an undo toast notification after AI-initiated changes, which are critical for building user trust in AI-mediated workflows and preventing unrecoverable errors.
3. **Receipt extraction feedback and correction.** The current receipt upload flow shows a camera/upload screen, but does not show what happens after the AI processes the receipt. Users should be shown the extracted items before they are added to the list, with the ability to correct errors, merge duplicates, or reject items. Without this, users cannot trust the AI extraction enough to rely on it.
4. **Substitution suggestions for out-of-stock items.** ShopBuddy already flags unavailable items with an "OUT" badge in the list detail view, directly addressing the pain point raised by P1 and P2. However, the design does not yet suggest what to buy instead. A future iteration should have the AI proactively recommend substitutes when an item is marked out of stock (e.g., "*Guacamole Tub is unavailable — substitute with fresh Avocados?*"), closing the loop and preventing users from having to figure out alternatives on their own while in-store.